

Code Architecture Design for a Multiplayer Game: Among Us

1. Game Overview

Among Us is a 2D/3D social deduction multiplayer game designed for 4 to 15 players. The game takes place in a closed environment (such as a spaceship) where players are randomly assigned one of two roles: Crewmates or Impostors.

Core Gameplay Mechanics:

- Movement & Collision: Real-time synchronization of player positions and physical boundaries.
- Task System: Interactive contextual mini-games distributed across the map for Crewmates to complete.
- Sabotage & Elimination: Impostors can trigger map-wide infrastructure emergencies or eliminate Crewmates directly.
- Voting & Discussion System: State-driven text/voice discussion rounds triggered by reporting a body or calling an emergency meeting, leading to player ejection based on consensus.

2. High-Level Architecture

To support this multiplayer loop seamlessly, the game architecture is divided into decoupled core subsystems that interact using a Server-Authoritative network topology.

- Network System: Handles low-level RPCs (Remote Procedure Calls), state synchronization, and connection payloads.
- Matchmaking & Lobby System: Manages session creation, private/public lobby states, connection limitations, and player preparation status.
- Player Management System: Spawns player entities, tracks authoritative local transformations, and updates cosmetics.
- Game State Manager: A centralized Finite State Machine (FSM) managing core phases: Lobby, Role Assignment, Live Gameplay, Emergency Phase, Voting, and Game Over.
- Task & Interaction System: Tracks dynamic completion vectors for team objectives and triggers mini-game interface components.
- UI & Voting System: Manages spatial canvas configurations, interactive voting rosters, countdown systems, and discussion panels.

3. Code Structure & Component Responsibilities

The codebase utilizes clear object-oriented structures in C#, separating networking concerns from immediate client-side visual states.

Core Classes and Structures:

- NetworkManagerManager: Custom implementation managing server lifecycle operations and structural scene changes.
- PlayerController: Handles local physical inputs and forces server reconciliation. Contains identity markers such as `IsImpostor` and `IsDead` synced via network properties.
- GameStateManager: Holds authoritative validation rules, evaluates win states, and handles transition triggers.
- VotingSystem: Receives client interaction commands, tallies the payload targets safely on the host, and distributes results globally.

Subsystem Communication Flow:

When an Impostor triggers a target elimination, the client invokes a secure command on the server. The server verifies position limits and role status. Upon successful validation, the server propagates a remote procedure call to all clients to change player visibility matrices and place the remaining interactive debris.

4. Design Decisions & Technical Reasoning

- Server-Authoritative Validation: All essential data states (roles, position checks, voting matrices) are held strictly on the host. This prevents injection-based exploitation and arbitrary state forcing by compromised clients.
- Finite State Machine Execution: Isolating the game phases using an explicit state machine model avoids conflicting logic loops, preventing interaction mechanics from running while active voting states are in progress.
- Data-Driven Task Interfaces: Tasks use a separate asset layer configuration. This separates structural interface states from spatial player logic, streamlining cross-platform asset compilation.